ChatGPT

Technical Architecture & Transition Plan (Android to KMP)

Project Overview & Vision

We are building **Daily**, a Bible meditations mobile app, with an initial focus on Android and a roadmap to later support iOS via Kotlin Multiplatform. The goal is to deliver a modern, engaging Android app that can evolve into a cross-platform solution without duplicating core logic. By leveraging Kotlin Multiplatform (KMP), we intend to reuse the app's core code across Android (and eventually iOS) to maintain feature parity and accelerate multi-platform support [1]. In line with the Product Requirements Document (PRD) vision, the app will combine daily devotionals, guided meditations, journaling, and community features in one seamless experience.

Key Objectives:

- **Modern Android MVP:** Launch a fully functional Android app using Kotlin and Jetpack Compose UI, following best practices (similar to Google's NowinAndroid sample which is "a fully functional Android app built entirely with Kotlin and Jetpack Compose" [2]). The app will implement core features outlined in the PRD (daily devotional feed, Bible reader, journaling, verse memory, etc.) for a **single-user** experience initially.
- **Robust Architecture:** Use a multi-module **clean architecture** (MVVM) with dependency injection (Hilt) and an offline-first design, ensuring maintainability and scalability. The app's codebase will be organized into logical modules (core layers and feature modules) to enforce separation of concerns and prepare for multiplatform support.
- **Kotlin Multiplatform Readiness:** Structure core business logic as pure Kotlin modules from the start so they can be shared with an iOS app. We plan a phased transition to KMP where after the Android MVP, the core logic modules are made **KMM (Kotlin Multiplatform Mobile)** compatible (e.g. using multiplatform libraries for database and networking). This will allow developing an iOS client that reuses these Kotlin libraries for data, domain, and logic.
- **Timeline & Roadmap:** We outline three phases of implementation, aiming for an iOS release (or beta) by **April 2026** (~6–8 months from now).
- *Phase 1 (MVP)* – Android app with core features, delivered in ~3–4 months (targeting Jan 2026).
- *Phase 2* – Feature enhancements on Android and groundwork for KMP, ~2–3 months after MVP (target ~Mar 2026). By the end of this phase, the app remains Android-only but the codebase is **KMP-ready** [3].
- *Phase 3* – Kotlin Multiplatform integration and **iOS app development**, along with a major **UI/UX redesign** to coincide with the multi-platform launch (targeting Apr 2026 for initial iOS rollout). This phase will deliver an iOS client (leveraging shared Kotlin logic) and refresh the app's design for both platforms.

Throughout these phases, we prioritize a **privacy-first, offline-capable** user experience as detailed in the PRD. The following sections detail the technology stack, module architecture, and each implementation phase.

Technology Stack & Architectural Patterns

Programming Language: Kotlin is used for all development. Kotlin offers first-class support on Android and is the basis for KMP, enabling code sharing with iOS in the future. All business logic will be written in Kotlin and placed in modules that can be made multiplatform.

UI Framework: Jetpack Compose is used for the Android UI layer, enabling declarative UI and rapid development. Compose is Kotlin-friendly and aligns with modern Android best practices (NowinAndroid is built entirely with Compose). Using Compose also positions us to potentially use **Compose Multiplatform** for iOS in the future, although our current plan is to use native SwiftUI on iOS with shared logic.

Architecture Pattern: We adopt a **Clean Architecture** approach with MVVM (Model-View-ViewModel) for presentation. The **ViewModel** layer (in Kotlin) will hold UI state and business logic for each screen, exposing state flows that the Compose UI observes. This separation allows us to reuse ViewModel logic on iOS by writing expect/actual or using KMP-compatible state management (e.g., `StateFlow` observed from SwiftUI) [4] [5]. The **data layer** uses repository patterns, and the **domain layer** contains use-cases and business rules, decoupling UI from data sources. This layered approach makes the core logic platform-agnostic and easily testable.

Dependency Injection: We will use **Dagger Hilt** for DI on Android to manage object creation and scope (e.g., providing singletons for repositories, database, etc.). Hilt will inject ViewModels and other dependencies in Compose. For multiplatform, we will design DI such that the shared code expects certain implementations to be injected (e.g., platform-specific implementations for network or storage if needed). Hilt is Android-specific; on iOS we may use manual DI or a service locator for the Kotlin components. The DI setup ensures loose coupling between modules.

Networking: Use a multiplatform-friendly HTTP client such as **Ktor Client** or Retrofit (with OkHttp for Android). Ktor Client is KMP-ready and can be used in shared code for API calls [6]. We'll start with simple REST endpoints (or Firebase services) for features like sync, content fetch, etc. Because our MVP is largely offline-first, network calls are mostly for sync and content updates.

Local Data & Offline: Adopting an **offline-first** strategy means the app will rely on local storage for primary data access, syncing in background [7] [8]. We will use a database (SQLite) via a multiplatform library **SQLDelight** for persistence, as it can generate Kotlin code for both Android and iOS [4] [9]. For instance, Bible text, devotional plans, and journal entries will be stored locally so the app is fully usable offline. The PRD emphasizes that core features (reading, journaling, plans, verse memory) work without internet [10]. SQLDelight will reside in a shared data module. On Android, we might start with Room for familiarity, but we plan to migrate to SQLDelight (or use it from the start) to ease multiplatform transition [11]. Data synchronization (e.g., uploading journal to cloud, downloading new devotional content) will be handled via simple cloud endpoints or Firebase. We'll likely use **Firebase Cloud Firestore** for cloud sync of user data and real-time updates (it has built-in offline caching) [12] [13]. Firebase Auth will handle user

accounts and authentication tokens on both platforms, with Google sign-in support on Android (and Apple sign-in on iOS) [14] [15].

Testing: Emphasis on unit testing for shared logic (e.g., verifying the verse memorization SRS algorithm, or journal encryption logic) on the JVM, since the core will be in Kotlin. ViewModels and use cases can be tested without Android dependencies. Instrumentation tests on Android for Compose UI and database integration will be set up (possibly using Robolectric or Jetpack Compose testing). Continuous Integration (e.g., GitHub Actions) will run the test suites to maintain quality [16].

Reference Implementation: The architecture is inspired by Android's **Now in Android** app which follows a multi-module clean architecture with Compose UI, Hilt, and modern practices. NowinAndroid organizes code into `core` modules (for data, model, UI theming, etc.) and `feature` modules for each screen/feature, using Compose Navigation to navigate between feature modules. We will emulate this structure to create a scalable codebase.

Modular Architecture (Modules and Layers)

To enforce separation of concerns and enable parallel development, the project is structured into multiple Gradle modules. This aligns with the plan from our prior research on feature modularization. The high-level module categories are:

- **App Module (`app`):** The main Android app module (entry point). This depends on all feature modules and orchestrates navigation. It also includes the Hilt application class for DI and overall app configuration.

- **Core Modules (`core:*`):** Reusable modules providing foundational logic, models, and services used across features. These will eventually become **KMP shared** modules. Our core modules and their responsibilities (as initially defined) include:

- `core:common` – Common utilities, extensions, and constants used app-wide [17].
- `core:model` – Data models (Kotlin data classes) shared across app layers (e.g., data transfer objects for Devotional, Plan, JournalEntry, User) [18].
- `core:domain` – Business domain layer, containing use case classes and interfaces for repositories (the abstraction of data layer) [19]. This defines operations like `GetDailyDevotionalUseCase`, `AddJournalEntryUseCase`, which the app can use without knowing data details.
- `core:data` – Data layer implementations of repositories, coordinating data from local and remote sources [20]. For example, `DevotionalRepositoryImpl` fetches devotionals from a local database or network API as needed. This module will depend on `core:model` and platform-specific data sources.
- `core:localdatasource` – Handles local storage concerns such as the database (SQLDelight/Room) and preferences [21]. It defines DAOs or database interfaces for other modules to use.
- `core:remotedatasource` – Handles network API calls (REST clients, Firebase integration) [22]. It will include service definitions (e.g., Retrofit interfaces or Ktor clients) and data mappers from network to model.

- *(Other core modules can be added as needed, e.g.,* `core:analytics` *if we separate analytics, or* `core:encryption` *for crypto utilities, etc.)*

- **Feature Modules (**`feature:*`**):** Each major user-facing feature or screen is encapsulated in its own module. Feature modules contain the UI (Compose screens), their own ViewModels, and any feature-specific logic. They depend on the core modules for data and domain. The list of planned feature modules (carried over from our previous module plan) includes [23] [24]:

- `feature:landing` – Onboarding/landing screens (welcome, sign-in/up flow) [25].
- `feature:authentication` – User auth flows (register, login, password reset) [26].
- `feature:home` – Main dashboard after login, which likely shows the Daily devotional feed and navigation to other sections [27].
- `feature:devotional` – (This could be part of home or separate) If needed, module for the daily devotional reading UI and related logic.
- `feature:bible` – Bible reader feature (scripture text view, search, highlight) [28].
- `feature:journal` – Encrypted journal UI (list of entries, editor screen) [29].
- `feature:memory` – (Referred as `feature:quotes` in legacy list or integrated into Bible) The verse memorization feature (list of memory verses, quiz UI) [30]. We might rename `quotes` to `memory` to clarify purpose.
- `feature:reading-plans` – Plans library and plan detail screens [31].
- `feature:prayer` – Prayer list/tracker (if included separately from journal) [32].
- `feature:community` – Social feed or Circles (small groups) feature [33].
- `feature:circles` – Possibly separate from general community: handles group chats, posts (if we break it out).
- `feature:church` – Church channels integration (joining a church, viewing church-provided plans).
- `feature:settings` – Settings UI (profile, preferences, notification settings) [34].
- *Additional modules from previous outline:* account/profile [35], AI (if AI chatbot was planned) [36], meditation (if separate from devotionals) [37], etc. Many of these are future or optional; for MVP some will not be implemented but placeholders can exist.

Each feature module contains its Compose screens, ViewModel(s), and any feature-specific data logic. They *only* interact with data via the core domain/repository interfaces, not directly to DB or network. This ensures replaceability and facilitates using the core logic on iOS. For example, `feature:journal` will use a `JournalViewModel` which calls `AddJournalEntryUseCase` (from core:domain) – the use case then uses the repository implemented in core:data to save to DB and sync, without the UI needing to know details.

- **Design System Module (**`designsystem`**):** A UI toolkit module with common **Compose UI components**, theming, and styles used across the app [38]. This includes colors, typography, reusable widgets (buttons, loading indicators), etc. Following NowinAndroid's approach, `designsystem` provides a consistent look & feel. It will allow easy global theme changes – important for the planned **redesign** in the future (we can update the theme in one place). For the redesign (planned in Phase 3), changes will concentrate in the designsystem (e.g., adopting Material 3 "Material You" design, new color palette, updated component styles). Both Android and iOS will follow the same design guidelines: on iOS we'll create analogous components in SwiftUI or use Compose Multiplatform if we go that route.

Module Dependencies: Each feature module can depend on `core` modules and `designsystem` (for UI components). They should not depend on each other directly to keep features modular. Navigation between features is handled through the `app` module (using a navigation graph or Compose Navigation routes that the app composes together). This decoupling allows us to include/exclude features easily and helps with parallel development. The **app** module ties everything together – it sets up dependency injection graph (providing instances of core components), configures navigation, and includes the entry Compose UI (e.g., NavHost).

All core modules will be packaged in such a way that we can include them in a Kotlin Multiplatform shared library. Concretely: - `core:model`, `core:domain`, `core:common` are pure Kotlin – straightforward to mark as commonMain sources for KMP. - `core:data` can be mostly common Kotlin, with expect/actual for platform specifics if needed (e.g., actual implementations using SQLDelight or Firebase). - `core:localdatasource` and `core:remotedatasource` likely have platform specifics (SQLDelight uses Kotlin/Native for iOS, Firebase SDK is not KMP-friendly directly). We may structure these as part of core:data but with expect/actual or separate them into platform-specific source sets. Alternatively, we can use multiplatform libraries (SQLDelight, Ktor) that abstract away differences so that even these modules compile for iOS. For Firebase (if heavily used), we might call it on Android side only and use a different approach on iOS (or integrate via KMP wrappers).

Legacy Modules: The old architecture reference included a `feature:presentation` legacy module [39]. This will be refactored away by moving code into the above feature modules. We'll incrementally migrate any old code (if we started from an existing codebase) into this new structure.

In summary, our modular setup mirrors the structure outlined in the sample technical doc [40] [41], providing clarity and enforceable boundaries. It ensures each feature is self-contained and the core logic is centralized for sharing.

Phase 1: Android MVP Implementation

Timeline: ~3–4 months (October 2025 – January 2026).

Goal: Deliver a functional Android app (APK) covering the core individual experience as defined in the PRD's MVP (v1.0) scope [42] [43]. This version targets Android only, providing value to users and a foundation for future expansion. We focus on single-user features and offline functionality, deferring complex social/cloud features to later phases.

Scope of MVP (v1.0): [44] [45]

- **Daily Devotional Feed:** Implement the home screen showing a daily devotional (scripture + reflection). For MVP we can include a starter set of devotionals (e.g., a built-in 30-day reading plan) stored locally [46]. The 5/10/20 minute length toggle might be simplified at first (possibly start with a single default length devotional to reduce initial content needs [47]). The feed shows today's devotion and perhaps recent or upcoming ones. Marking a devotion as completed and tracking streaks should be included (basic habit formation metric). This is the centerpiece of daily use.

- **Bible Reader:** Integrate a Bible reading interface to allow reading full chapters and quick verse lookup [48]. Include one translation (KJV, which is public domain) fully offline in the app [49]. Users can navigate by book/chapter, search for verses (basic text search). We'll implement a simple UI with chapter selection and scrolling text. Advanced features like parallel translation or extensive footnotes are not MVP (can show just reference links or none). Ensure the text is stored in the local DB and loaded quickly. Also allow copying verses or simple highlighting (optional in MVP).

- **Encrypted Journaling:** Provide a **Journal** screen where users can create, view, and delete journal entries [43]. Enforce **client-side encryption** for entries – e.g., derive an encryption key from user's password or a device-keystore key if using guest mode [43]. For MVP, a simple encryption (we can use Android's Jetpack Security Crypto or SQLCipher library) will be implemented to prove the concept. The journal entries are stored locally (SQLite) and synced to cloud only in encrypted form if user is logged in. Basic features: add entry (with timestamp, optional title), list entries, view entry. Search within journal could be rudimentary or omitted in MVP (given encryption makes searching tricky – maybe allow filtering by date or tags if implemented). The focus is privacy: we will highlight that the journal is secure and personal. If possible, allow an app-level PIN/biometric lock for the journal section (this could be a nice-to-have if time permits, otherwise Phase 2) [50] [51].

- **Verse Memorization (Memory Verses):** Implement the ability for a user to save verses to a "memory list" and practice them with a basic spaced repetition mechanism [52]. MVP might keep it simple: the user can mark a verse to memorize (from Bible reader or devotion screen). Those verses appear in a "Memory Verses" list. The review mechanism can be basic flashcards in MVP: show the verse text with hidden parts or just prompt the user to recall and then show answer [52]. We will schedule review notifications using a fixed interval (e.g., everyday for a week, then weekly) as a placeholder SRS algorithm [53]. Tracking which verses are due for review each day and showing a count or notification ("You have 3 verses to review today") is a stretch goal in MVP – at minimum, the user can manually review their saved verses. This feature differentiates us even in MVP, so we will include the bare essentials for it.

- **Audio – Text-to-Speech:** Provide an option to **listen** to content via TTS for both scripture and devotionals [54]. On Android, we'll use the built-in Google Text-to-Speech engine. MVP UI: a play/pause button on the devotional and Bible screens that reads the text aloud. We don't need complex audio controls beyond play/pause and maybe a speed adjuster. Ensure it works offline: instruct user to download the voice data for offline TTS (we can detect if no TTS engine or data is present and prompt accordingly). This aligns with the PRD goal of having all content available in audio form from day one [55]. Background playback (audio continuing when app is in background) can be supported via Android media APIs, but if time is short, we can allow audio only when app in foreground for MVP. Ideally, we'll integrate Android's MediaSession so playback can continue with screen off and show a notification player.

- **Offline-First Functionality:** Core features must work fully offline from MVP launch [56]. Concretely:

- All content needed for MVP (the default devotional plan, KJV Bible text, etc.) will ship with the app or be downloadable on first use. The local database will be pre-populated with devotionals and Bible text (or included as assets).
- Any user data (journal entries, memory verses, etc.) is stored locally first, and if the user is online and logged in, sync it to cloud in background. If offline, queue the sync for later [8].

- The app should not block or show "no connection" for core usage – e.g., if it's offline and it's a new day, the app should show the devotional for that day if it was preloaded or show yesterday's with a notice if new one can't be fetched [57] [58] .
- We will implement a simple sync mechanism: possibly using Firebase Firestore for journal and settings backup (since Firestore automatically syncs when connection is back) [56] . Alternatively, for MVP, we could even skip cloud sync entirely and just allow an **optional backup** (e.g., export journal to Google Drive manually) to avoid server work – but having at least basic Firebase sync will enhance the experience. The PRD suggests even a simple encrypted file backup to Drive or iCloud could suffice for MVP [59] .

- Testing offline scenarios will be part of QA.

- **Accounts & Authentication:** Implement basic email/password signup and login (using Firebase Auth or a simple custom auth). **Guest mode** will be allowed so a user can use the app without creating an account [60] . However, having an account enables data sync across devices and will be required for future social features. For MVP, we likely allow skipping login entirely, letting the app generate a local profile (with a unique ID) so user can use all features offline. We'll provide an option in settings or onboarding to create an account to enable cloud backup. If using Firebase Auth, it's straightforward to integrate Google Sign-In as well for convenience (on Android). The app must handle an upgrade from guest to logged-in (merging local data to cloud).

- **Onboarding Flow:** On first launch, after a welcome screen (feature highlights), prompt to sign up or continue as guest [61] [62] . Then ask for initial preferences: preferred devotion length (5/10/20 min, etc.), daily reminder time, and optionally their church or a general interest selection [63] . However, to keep MVP tight, we can shorten onboarding: maybe just set reminder time and let them skip other steps. The daily reminder notification for devotion is a key habit-forming feature, so likely in onboarding we will ask permission to schedule notifications at user's chosen time (e.g., 7 AM) [64] .

- After onboarding, user lands on the Home feed showing the first devotional.

- **Basic Navigation & UI:** Use a single-activity architecture with Compose Navigation or a manual navigation system. Likely a bottom navigation bar (tabs) for main sections: e.g., *Home*, *Bible*, *Journal*, *Plans*, *More*. The Home tab shows devotionals, Bible tab opens reader, Journal tab opens journal, Plans tab can list reading plans, More tab for settings/profile. If too many features, we might opt for a Navigation Drawer or just show fewer tabs in MVP (maybe Home, Bible, Journal, More; and within More we include Settings, etc.). Navigation between Compose screens will be handled in the app module via NavHost. Each feature module provides composable screens that register routes.

- **UI/UX Design:** For MVP, we'll implement a **simple, clean UI** using our design system components. It should look modern but we won't spend too much time on fancy custom graphics. We'll follow Material Design 3 guidelines, using Compose Material components themed appropriately. We have dark mode support out-of-box with Compose Material. We'll ensure font sizes and contrast are adequate (consider accessibility). The design will likely evolve, but MVP's focus is functionality and clarity. We can incorporate imagery like a subtle background image on devotionals or a splash screen illustration if readily available, but it's optional. A consistent color scheme (possibly a soothing blue/teal palette to give a calm, spiritual feel) will be applied via `designsystem`.

- **Quality & Monitoring:** Before release, we'll do internal testing (and possibly a closed beta). We will integrate **Crashlytics** for crash reporting so we catch any runtime issues early (particularly important for offline scenarios and encryption). We'll also instrument basic analytics (Firebase Analytics) to measure engagement (this will help verify metrics like daily active users, but user privacy is considered – we track generic events, not content of journal, etc.).

Deferring to Later (Post-MVP): We intentionally **exclude or simplify** certain things in MVP to meet the timeline: - **Small Group Circles and Church features:** No social or group chat in MVP – it's single-player only [65] [66]. We won't implement Circles, friend invites, or church channels yet. This avoids needing a complex backend and moderation. (We will lay groundwork in data models to not paint ourselves into a corner – e.g., design the data schemas with potential user IDs and group IDs, but UI is hidden.) - **Content variety & volume:** MVP will use mostly static content (the initial devotionals and KJV Bible). We avoid dependency on external content APIs or licensing issues initially [67]. All included content is either original or public domain (no need to license NIV yet, etc.). This keeps costs down and development simpler (as noted in PRD: MVP content relies on freely available sources [68]). - **Payments/Monetization:** No in-app purchases or subscriptions in MVP [69]. The app is free at start; we won't integrate Google Play Billing until Phase 3. Monetization strategy will kick in later (Premium tier, etc.), after we have user traction. - **Advanced security features:** While we implement core encryption for journal, we might not implement features like 2FA, or complex encryption passphrases in MVP due to time. The basics (secure storage, encryption at rest) will be covered, but e.g., the user might not be given the option to set a custom encryption password in MVP (we can simply use device keystore and note that if they want true zero-knowledge, that's coming later). - **Secondary features:** Verse sharing (to social media), verse art images, comment threads on devotionals, advanced search with fuzzy matching, custom plan creation, etc., are all beyond MVP scope. If the basic set is done early, we can consider one or two small extras (maybe allow sharing a verse or devotional via the Android share sheet as text image). But the plan is to **focus on core use cases**. - **iPad/ Tablet or Web:** Not applicable in MVP. The Compose UI will naturally scale to different screen sizes, but we'll optimize for phones in portrait. Tablet UI can be refined later. No web app in MVP (though content might be managed manually via Firebase or JSON).

By the end of Phase 1, we expect to have: - The **Daily App** running on Android, fulfilling the primary spiritual use cases (daily reading, journaling, memorization) offline. - A codebase structured into modules, albeit the KMP aspect may not be fully implemented yet, but the separation is there. We will likely keep all code in the Android project during MVP, but plan the refactor to KMP by ensuring pure Kotlin logic is not tied to Android frameworks. For instance, ViewModels might use `androidx.lifecycle.ViewModel` in MVP; when going KMP, we might replace those with KMP ViewModel alternatives or keep them in common code via libraries like `KMPViewModel`. We remain mindful of this but may not actually pull the trigger on multiplatform until Phase 2. - Preliminary analytics on usage and feedback from early users to inform Phase 2 improvements.

MVP success criteria (as per PRD) include launching to a small user group and seeing that they engage daily and that offline use is smooth [70]. Technically, success means no major crashes, data correctly saved offline, and positive reception that the app is easy to use and beneficial.

Phase 2: Enhancements & Kotlin Multiplatform Preparation

Timeline: ~2–3 months post-MVP (February – March 2026).

Goal: Improve and expand the Android app's features based on MVP feedback, and refactor the codebase to be fully **Kotlin Multiplatform**-enabled in preparation for iOS development. By the end of Phase 2 (v1.1 of the app), the Android app should feel more polished and content-rich, and the core logic should be running in a shared module that can be consumed by an iOS project. The app is still Android-only during this phase, but with KMP in place "under the hood" (the iOS client will come in Phase 3).

Scope of Phase 2 (v1.1) – "Enhanced Content & Social Foundations" [71] [3] :

1. **Broader Content & Features:**
2. **Add More Biblical Content:** Include an additional Bible translation (or two) based on user feedback [72] . For example, if many found KJV hard to read, add the **World English Bible (WEB)** which is modern English and public domain [72] . We can make WEB available as an option in settings or even default to it for new users. This addresses an immediate user need without licensing cost. We may also integrate a free API for a modern version (like ESV via Crossway API) for online use as a stopgap [72] . If technically simple, the app could fetch verses from ESV API when online (and not store offline to respect terms), giving an alternative translation experience.
3. **Expanded Devotional Library:** Provide more variety in devotionals. By v1.1, we can introduce a **Plans Library** section [73] . This feature will list multiple reading plans (topical or biblical). For instance, alongside the daily default plan, include a few curated plans: e.g., a 7-day "Beginner's Prayer Journey," a 14-day "Forgiveness Study," or an imported public-domain devotional (like Spurgeon's *Morning & Evening* for a month) [74] . Users can browse plans, start one, and track their progress in a new **"Plans" screen**. This requires implementing UI for plan lists and plan detail (with day by day content). Under the hood, we likely add a `PlanRepository` and related data classes. We may reuse the existing daily devotional logic to display plan entries. The key new function is allowing multiple concurrent plans and tracking each separately (maybe a simple progress percentage or day X of Y shown) [75] .
    - *Note:* The PRD suggests enabling multiple ongoing plans and friend sharing by v1.1 [75] . For now, we'll allow multiple plans per user (no restrictions), but not yet implement friend sharing (we'll prepare the data model so a plan can have an "invite code" or similar in the future).
4. **Notifications & Reminders:** In MVP we had a fixed daily devotion reminder. In Phase 2, make notifications more configurable and comprehensive [76] . Add a settings UI where users can turn on/off reminders or set a different time. Introduce **plan reminders** (if a user hasn't completed today's reading, send an evening reminder), **streak celebrations**, and memory verse review reminders [77] . Also, since we might quietly start building backend features, integrate Firebase Cloud Messaging for any server-push notifications if needed (e.g., later social notifications) [3] . For now, most notifications can be scheduled locally. This enhancement keeps users engaged and lets them personalize their habit triggers.
5. **UI/UX Polish:** Address any usability issues from MVP feedback. Possibly implement an **intro tutorial** or tooltips for new users (e.g., highlight "Tap the star to save a memory verse!"). Ensure the **Settings** screen is fully built out with toggles for notifications, account management (change password, etc.), and an **Export Data** option (allow user to export their journal to a text file or backup – showing our transparency) [78] . Also, ensure accessibility improvements (larger text support, etc.) are in place by now [77] .

6. **Social Foundations:** While full social features (groups, etc.) come in Phase 3, Phase 2 can lay groundwork:

    - Create **username/display name** fields in the user profile (so that later, friends or group members have an identifier beyond email) [79] . If the user is logged in, allow them to set a username (and maybe auto-generate a random one if not). This will prepare us for friend invites in v2.
    - Possibly implement a hidden feature to **share a reading plan link**: e.g., user can tap "Share this plan" and the app generates a link or code that, if another user opens, simply opens that plan in their app (or invites them to it) [79] . This is an intermediate social step to test content sharing without building full friend lists. It basically mimics "read the same plan with a friend" by manual coordination. We won't yet have an in-app friends list or group chat for it – they would have to discuss outside the app. But we can record internally if two users joined via the same link (maybe a proto "shared plan" concept).
    - On the backend side, start designing the data model for **Circles** (small groups): e.g., a Firestore collection or a table structure. We might not expose any UI, but having an initial model means that if we have any enthusiastic test group, we could manually create a small closed beta group and test posting. However, likely we keep it dark in UI until v2.
    - We will **enforce login** for cloud sync now (in MVP it was optional). In v1.1, if user wants their data across devices, encourage them strongly to create an account [78] . We may prompt guest users after some time to sign up ("Create an account to save your progress to the cloud"). This solidifies our user accounts in preparation for social features which will require accounts.

7. **Technical Enhancements:**

8. **Refactor to Kotlin Multiplatform:** Phase 2's crucial development task is to modularize and convert core code to a **shared KMP library**. We will create a new Gradle **KMM module** (for example, `shared` or repurpose our core modules as KMP). Likely approach:

    - Combine or mark `core:model`, `core:domain`, `core:data` as Kotlin Multiplatform modules with common code. We will move any Android-specific implementations (Room, Firebase) to the Android source set, and create iOS stubs or expect/actual as needed [4] [9] . For instance, use SQLDelight so the database logic is common, with platform drivers injected at runtime.
    - The shared module will produce an Android library for use in the Android app and an iOS framework for the future iOS app [80] .
    - **Networking:** Replace Retrofit (if used) with Ktor Client in shared code so that networking works on iOS. Or use Retrofit on Android and plan to use a different approach on iOS by abstraction. Ideally, adopt one strategy: Ktor client in common code with OkHttp engine on Android and URLSession engine on iOS. That way our `RemoteDataSource` can be multiplatform.
    - **Data Sync & Firebase:** Firebase SDK is not multiplatform. We have options: keep Firebase calls on Android side only for now, or integrate something like `Firebase KMP SDK` (community libraries exist) or use REST APIs for Firebase. A simpler path: for Phase 2, we might limit cloud operations to Android and not require them on iOS until launch. But since by Phase 3 iOS should sync too, we plan ahead. Perhaps use a lightweight custom REST

backend for critical sync (e.g., implement a small Ktor server for journal sync) which both platforms can call. Or use Firebase Auth's REST API to sign in on iOS side. This requires evaluation; given time, we might decide to initially not include full Firebase in shared code. We could have expect/actual in `core:remotedatasource` where Android actual uses Firebase SDK, and iOS actual uses a placeholder or minimal implementation (like hitting Firestore via REST or deferring sync until we find a solution). However, the PRD suggests using multiplatform-ready solutions like **SQLDelight for DB and treating sync as secondary** [67] [81]. So an approach: rely on the local DB as source of truth on iOS too, and do periodic sync via simple HTTP calls.
    - **ViewModels:** We need to consider how to share state with iOS. Options: (a) Use KMP libraries like **Kotlin MVVM** or `mokoMVVM` that provide multiplatform ViewModel and state flow that SwiftUI can observe; or (b) maintain separate ViewModel on iOS side that calls shared use-cases. A middle ground: keep business logic (use cases, repositories) in shared, and let Android have Android ViewModels wrapping those, iOS will have SwiftUI ViewModels (ObservableObjects) that call into shared logic. For Phase 2, we might choose to simply share use-case classes and repository implementations, and not literally share ViewModel classes. That means minimal changes to the Android code – we keep Android ViewModel classes (which may just delegate to shared use cases). On iOS, we will later write SwiftUI view models that use those same use-case classes via the KMP framework. The PRD mentions an approach: use StateFlows in shared ViewModels and have SwiftUI observe them [82], which is elegant but requires making ViewModels in shared code. We might attempt that for completeness: e.g., use `Kotlinx.coroutines` flows and `FlowWrapper` for SwiftUI.
    - **Testing KMP:** Once refactored, we ensure all unit tests still pass now running on the common code. We might add iOS unit tests (Kotlin/Native tests) for core logic as well at this stage.
    - By end of Phase 2, **"core logic is KMP ready for iOS"** [3] – meaning one could drop the shared library into an Xcode project and have all data/domain functionality working.

9. **Performance & Polish:** Address any performance issues (maybe preloading data on background threads to avoid UI jank). If large lists (plans library, etc.), use Compose LazyColumn efficiently. Also implement any missing **error handling** from MVP (e.g., show a user-friendly error if a sync fails or if content not found). Possibly incorporate a loading indicator for network ops or when switching plans.

10. **Backend/Foundation Work:** If not already done, Phase 2 is a good time to set up some simple backend services needed for Phase 3:

    - If we plan on implementing friend invitations or sharing by Phase 3, set up a minimal cloud function or service to handle those (or decide to piggyback entirely on Firebase dynamic links).
    - Prepare push notification setup for iOS (APNs credentials in Firebase, etc.) so that by iOS launch we can send notifications cross-platform [83].
    - If any content management improvements: maybe create a script or simple admin interface to upload new devotional content to Firebase so that we can update the app's content remotely without app updates. This is not user-facing but helps us manage content growth (for instance, adding new plans or devotionals easily).

11. **Redesign Kick-off:** Start planning the **UI redesign** while implementing new features. Gather feedback on current UI/UX from MVP users. By end of Phase 2, we should have a design prototype

for a refreshed look that we will implement in Phase 3. This redesign may align with adopting Material You styling, new color themes, updated logo/branding, etc. Implementation will mostly happen in Phase 3, but Phase 2 can begin gradually – for example, update the app icon, or incorporate any quick wins like better typography from Material3.

Outcome of Phase 2: The Android app (v1.1) will feel more mature – more content (multiple plans, another translation), more user control (notifications, settings), and slightly stepping into social (usernames, plan sharing). Importantly, under the hood, we will now have the **shared Kotlin code** solidified. The core logic can run on iOS, and we might even start an experimental iOS project to verify integration (e.g., build a simple SwiftUI list of devotionals using the shared module). The PRD expects by v1.1 that we are "still Android-only, but core logic is KMP ready for iOS" [3], which is exactly our deliverable.

This sets the stage for Phase 3, where we will build out the iOS app UI and further expand features like full community groups and monetization.

Phase 3: Kotlin Multiplatform Integration, iOS Launch & Redesign

Timeline: ~3–4 months after Phase 2 (April 2026 and beyond).

Goal: Transform the project into a true multi-platform product by launching an **iOS version** of the app using the shared Kotlin code, and implement a comprehensive **UI/UX redesign** to coincide with the iOS release (and the introduction of community features and monetization). Phase 3 corresponds to a major version bump (v2.0), where the app evolves from an "Android MVP" to a cross-platform app with social/community functionality and the beginning of the business model (subscriptions).

Scope of Phase 3 (v2.0) – "Community & Growth Expansion" [84] [85]:

1. **iOS Application Development:**
2. Start development of the **iOS client** in Swift (SwiftUI for UI). Thanks to the KMP-shared modules (from Phase 2), the iOS app can call into the same `ViewModel` logic or use case classes to get data, ensuring consistent behavior and feature set [86].
3. **Architecture on iOS:** Likely use SwiftUI + Combine (or KMP's Flow via Combine interop) for reactive UI. We will create SwiftUI views for each feature (DevotionalView, BibleReaderView, JournalView, etc.) analogous to Compose screens. For each, the underlying data comes from the Kotlin shared layer:
    - We may instantiate shared ViewModels (if we made them multiplatform using state flows) and observe their `StateFlow` via `ObservableObject` wrapper. Alternatively, we use the repository use-cases directly in SwiftUI and manage state in SwiftUI. There are examples of KMP apps where the shared layer provides a `Flow` and SwiftUI subscribes to it.
4. **UI Design on iOS:** We will aim for a native iOS feel **while keeping consistent branding**. Colors, icons, and general aesthetics remain the same as Android (especially after the redesign). But we will adhere to iOS platform conventions for navigation and controls. For example, use a TabView for bottom tabs, NavigationStack for in-page navigation, iOS-style toggles in settings, etc. If Compose Multiplatform for iOS is stable by this time, we could consider using it to share UI code as well [4], but likely we stick to SwiftUI for a truly native experience unless resource constraints push us to reuse Compose UI. (The PRD suggests possibly using SwiftUI or Compose Multiplatform for iOS UI

[87] ; we'll evaluate stability – as of 2026 Compose Multiplatform may be viable, but it's new, so safer to do SwiftUI.)

5. **Feature Parity:** Ensure all features available on Android (post-Phase2) are available on iOS. This includes: showing the daily devotion feed, Bible reader, adding journal entries (with encryption – we might need to implement encryption on iOS via Kotlin code or use CommonCrypto), memory verses, plan library, notifications for iOS, etc. We will leverage as much shared logic as possible:
    - The encryption, SRS algorithm, etc., all come from shared Kotlin, so it's consistent.
    - For local storage on iOS, SQLDelight will use an iOS SQLite database for the same schema, so it "just works" given correct initialization.
    - Networking calls from shared code will use Ktor with an iOS HTTP client engine. Or if we integrated Firebase, we might need to use alternative means on iOS. Possibly we skip full Firebase login on iOS MVP by requiring users to use email/password (which KMP can handle via REST to Firebase Auth endpoints). This may require writing some platform-specific code in Kotlin (actual for iOS using Firebase REST API or using KMP library like FirebaseKMP).
    - Push Notifications: Set up APNs via Firebase Messaging or native iOS notifications for daily reminders, etc. The iOS app will prompt for notification permission and schedule local notifications just like Android (for daily devotions, plan reminders). For push (e.g., if we send a notification from server), integrate Firebase Cloud Messaging's APNs configuration. The shared code can define the message content, but actual registration for push and handling must be done in iOS code. We might need to implement a small iOS-specific push handler that calls shared logic when a notification is tapped (to navigate to appropriate content).
6. **Testing iOS:** We will do a TestFlight beta perhaps with the same pilot users to get feedback. Focus on ensuring the shared code works well under Kotlin/Native (fix any memory management issues, threading, etc., that differ from JVM). Also ensure that encryption works on iOS (key handling in Keychain if needed).

7. According to plan, by end of this phase or shortly after, we aim to **release an iOS version (even if beta)** [88] . This effectively doubles our reach and caters to mixed-platform communities.

8. **Community Features (Social Expansion):**

9. **Launch Small Groups (Circles):** Fully enable the **Circles** feature for social interaction [84] . Users can create or join circles (invite-only groups) within the app. This involves:

    - UI: A new **"Community" or "Groups" section**. For example, a tab or a screen listing the user's Circles. From there, they can open a Circle (group chat feed). Provide a "Create Circle" flow (name the group, get an invite code or link to share) [89] . Also a way to join an existing circle via code or link.
    - Messaging: Implement basic group chat within a circle [89] . Likely use Firebase Firestore or Realtime DB for messaging to avoid building entire infra. We'll integrate something like Firestore where each circle has a collection of messages. Use listeners to live-update new messages. The shared Kotlin code can provide a `ChatRepository` that wraps Firebase calls (on iOS, possibly use a lightweight KMP Firebase wrapper or call via REST API or just limit realtime features on iOS initial launch by requiring manual refresh – but preferably find a way to integrate Firebase realtime on iOS too, perhaps through multiplatform libraries or by writing minimal Swift code that calls Firestore SDK and passes data to Kotlin).

- Features: Text posts, maybe emoji reactions. We'll skip advanced things (no images or voice messages in v2, to minimize complexity) [90] . Ensure notifications for new messages (using FCM/APNs).
- Moderation: The circle creator can remove members or delete messages if needed. Basic UI for that (long-press message -> delete if owner). Possibly an "Report" button to admin (which might just send us an email for now).
- Integration: Circles tie into existing features:
- If members of a circle are on the same reading plan, show a discussion thread for that plan (the PRD described comments on plan days visible within the circle) [91] . For v2, a simpler approach: after reading a devotion, user can go to the circle and manually post about it. Automated threading can be a future improvement. But we might indicate what plan day a post is about if user chooses.
- Verse sharing to circle: allow user to share a verse or journal snippet to their circle directly from those screens [92] .
- Circle members can see each other's progress or streaks optionally (this may be a minor feature, maybe a toggle "share my activity to circle" – if enabled, show "Alice completed Day 5 of Plan X" in the circle feed or such).
- We expect that by adding Circles, we significantly boost engagement and retention (community effect). We will monitor metrics like number of messages and circles created.

10. **Introduce Church Channels (Beta):** As per plan, onboard a few pilot churches to test the **Church Channel** feature [93] .

    - Provide an interface (possibly a simple web portal or even manual JSON) for a church admin to create a channel, upload devotional plans or announcements, and generate a join code. On the app side, add UI for users to search or enter code to join a church channel [80] [94] .
    - Once joined, the church channel content appears in the app: e.g., a Church feed separate from personal feed, or perhaps integrated in the home feed (maybe a section "From Your Church"). For simplicity, we may give channels a section in the app (like how YouVersion has "My Church" section). But a minimal approach: the channel basically behaves like a special circle run by the church (with potentially unlimited members). The church can post reading plans (which appear like recommended plans to channel members) and announcements (which could appear as notifications or in a channel feed).
    - Implement minimal admin features: likely not building an in-app admin UI in mobile for church leaders. Instead, maybe a basic web admin or simply instruct pilot churches to send us content that we upload to Firestore. However, since we plan to scale it, we might implement a very basic web form or reuse Firebase console for content storage.
    - The **analytics** for church (if any in v2) might not be fully built but we can promise to send them some usage stats manually.
    - This feature is beta and mainly to prove out the concept and gather requirements. So we might work closely with 1–3 churches.
    - It's also a selling point for B2B model, which we plan to monetize (likely in future phases or late v2).

11. **Monetization Launch (Freemium Model):**

12. By v2, we plan to introduce the **Premium subscription tier** to start generating revenue [95] [96] . The timing is appropriate because by now we have a robust set of features and content. The monetization strategy from PRD will be implemented as follows:

    - **Implement Subscriptions:** Integrate **In-App Purchases** on Android (Google Play Billing) and iOS (StoreKit) for a Premium subscription (monthly/yearly) [97] [98] . Use the same product IDs across platforms where possible. Possibly use RevenueCat or Firebase Extensions to unify subscription handling, or do platform-specific logic and have our backend verify receipts.
    - **Paywall & Upsell:** Identify features/content to put behind the Premium paywall. As per earlier discussion [99] [100] and PRD:
    - Additional Bible translations (licensed ones like NIV, ESV) – by v2 we aim to have at least one such license, e.g., NIV. Those will be **Premium-only** content to justify subscription (because we incur licensing cost) [101] [102] . Implementation: in app, free users see NIV in list but tapping it shows a modal "Subscribe to access NIV and other modern translations" [103] [104] .
    - Premium **audio content** – e.g., human-narrated audio (if we manage to license or produce any by then) or higher-quality TTS/ambient sounds as add-ons [105] [106] . Possibly by v2 we include one professional audio (maybe an audio Bible for New Testament). Make that premium.
    - **Exclusive devotionals/plans:** If we have any content partnerships by then (or even classic content curated nicely), mark some plans as Premium [107] [108] . E.g., a 30-day study from a well-known author might be premium-only.
    - **Social/Community perks:** Possibly allow free users to join only a limited number of circles (say 1 circle) whereas Premium can join/create unlimited circles [109] [110] . This encourages heavy community users to subscribe but still lets casual users participate a bit. We must be careful not to cripple core community, but a limit like 1 free circle might be acceptable as PRD suggests [109] .
    - **Device sync:** If in MVP we allowed free sync, we might decide to require login+premium for multi-device sync. But basic backup should probably remain free to not risk data loss. We could differentiate: free account = sync journal and basics; premium = sync across unlimited devices *plus* maybe web access if we ever have web.
    - **UI Customization**: not critical, but perhaps Premium users get extra themes or icons (small perk).
    - We will **not** put core daily devotion or Bible reading behind paywall – those remain free to keep the ministry ethos. The idea is premium adds convenience and depth (versions, audio, exclusive content), while free covers basic spiritual needs.
    - **Build Paywall Screen:** When user hits a premium feature, show a nice screen highlighting benefits of Premium (perhaps 3-column list of features, "Free vs Premium"). Include the pricing and a CTA to subscribe. Also possibly an offer like a free trial for 7 days (both stores support trial periods).
    - **Backend for Subscriptions:** Implement verification (like using Play Store purchase tokens and App Store receipts). Possibly use Firebase Functions to verify receipts server-side to prevent fraud. Or simpler: trust client for now but plan to secure later. We'll also implement logic to handle cancellation (maybe grace period access).
    - **Promotion:** We might give early bird pricing or founding member discounts as PRD discussed, but those are marketing details. We ensure the app gracefully downgrades if sub expires (no data loss, just lock premium content again).

- As noted in PRD, communicate transparently that premium supports the app's mission and not just greed [111] [112]. Possibly include a note or onboarding slide about this to keep user trust (e.g., "Your subscription helps us license more translations and create more content to bless others").

13. **B2B Monetization (Church Plan):** Not necessarily fully launched in v2, but we explore it:

    - If our pilot churches loved the channel feature, we can prepare a **Church Plan offering**. Maybe we decide on pricing like $X per month for a church (depending on size) [113] [114]. We likely won't automate payments for this in v2; instead, we might sign one church on a contract or free trial and plan to scale it in future.
    - The concept: a church pays, and all its members get premium for free (subsidized) [115] [116]. We can implement that by flagging users in that church's channel as premium-entitled. For now, since only a couple churches, we might manually upgrade those users or use a simple mechanism (e.g., a "churchCode" that unlocks premium).
    - We'll refine how to integrate this in the app UI (perhaps show "Provided by Your Church" if a user's church has a subscription).
    - No robust payment portal for churches in v2 – that would come later with more dev resources or when scaling.

14. **UI/UX Redesign Implementation:**

15. We coordinate a **major redesign** in Phase 3 to refresh the app's look and address any UX shortcomings observed. Typically, after getting MVP out and initial feedback, it's ideal to refine the design for broader release (especially to make a splash on iOS launch).
16. Redesign elements may include:
    - **Visual theme update:** Possibly adopt Material You on Android (dynamic theming, rounded shapes, etc.) and analogous styling on iOS. Update color scheme, typography for a more modern, distinctive look. Ensure a consistent brand identity across platforms (maybe a new logo or refined iconography).
    - **Navigation reorganization:** Based on usage, we might adjust what's easily accessible. E.g., if Journal wasn't used as much, perhaps demote it from tab bar; if community is key, give it a prominent tab. If adding Church, might incorporate that prominently.
    - **Higher fidelity UI components:** More custom illustrations or backgrounds to make the app feel polished (e.g., a nice graphic on the welcome screen, or subtle background gradients).
    - **Better onboarding:** The redesign might include an improved onboarding flow with nicer graphics, or an in-app tutorial for new social features.
    - **Consistent design language:** Because now we have two platforms, define a design language that works on both (color, fonts, icon style). Android and iOS will each use native components but we ensure the overall feel is the same app. For instance, use the same color codes and similar layout concepts (tab bar on iOS vs bottom nav on Android).
    - **Accessibility review:** The redesign should also incorporate any needed changes to make the app more accessible (ensuring color contrast, button sizes, etc., are optimal).
17. Implementation will largely occur in the `designsystem` module for Android (e.g., replacing theme definitions with Material3, updating component styles) and similarly adjusting SwiftUI view styles on iOS. We should try to time the redesign so that both platforms launch with the new look (to avoid one looking outdated).

18. Possibly engage a UI/UX designer during this phase if we have not already, to get professional design input, given this is a crucial public launch stage.

Outcome of Phase 3: By April 2026, we aim to release **Daily App 2.0**: - **Android 2.0:** With community features, refined UI, and subscription model active. - **iOS 1.0:** The first iOS release, feature-equivalent to Android and with the same redesign, likely marked as version 2.0 as well for parity. - We will likely label it a major update and do a marketing push (perhaps around Easter 2026, which is in April – a good time for a faith app launch). - The ecosystem now covers both major mobile OS, enabling friends and church groups regardless of device to participate (which was crucial for growth, as noted: “many Christian communities have mixed device usage” [117]). - We’ll also begin measuring revenue and seeing how users respond to premium. The app’s success metrics (engagement, retention, MTS/W as PRD calls it) should see an uptick due to social and new content, and we’ll track conversion to premium.

Finally, beyond April 2026, further phases (v3+) would focus on scaling and optimizing: - Expanding internationally (add more languages/translations), - Advanced features like AI-driven recommendations or integration with wearables, - Possibly a web companion or more church admin tools, - Continuous improvement based on analytics (e.g., if memory feature is underused, revamp it; if community thrives, invest more there), - etc., as outlined in future ideas [118] [119].

For now, we have a clear path to deliver a high-quality Android app and then transition into a multiplatform product, all while adhering to the product’s core vision and values (ensuring technology serves the spiritual purpose, not the other way around).

Attachment: Product Requirements Document (PRD)

(The following is the full Product Requirements Document for the Daily Bible Meditations App, which provides detailed context on the product vision, target audience, features, and roadmap. It has been included for reference.)

[120]

[121]

[122]

[123]

[124]

[125]

[126]

[127]

151

152

153

154

155

156

157

---

1 3 4 5 6 7 8 9 10 11 12 13 14 15 16 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56
57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86
87 88 89 90 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107 108 109 110 111 112 113 114
115 116 117 118 119 120 121 122 123 124 125 126 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141
142 143 144 145 146 147 148 149 150 151 152 153 154 155 156 157 8fa63dbd-a740-4a9e-ac92-378defffb824.md

file://file_0000000059606230914e4cf2ae99007d

2 GitHub - android/nowinandroid: A fully functional Android app built entirely with Kotlin and Jetpack Compose

https://github.com/android/nowinandroid

17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41
968bfae5-8705-48ba-aa93-26ddc0ff0a77.md

file://file_000000003a30620998f9eb2247346fa6